Lec-20: Containers, Iterators, and Algorithms

Associative containers overview

Associative containers store elements in **sorted order** (by key). Internally implemented as **red-black trees** — all operations are $O(\log n)$.

- `std::set<K>` — sorted collection of *unique* keys
- `std::map<K,V>` — sorted key–value pairs, unique keys
- `std::multiset<K>` — sorted collection allowing *duplicate* keys
- `std::multimap<K,V>` — sorted key–value pairs, duplicate keys allowed

Keys must support `operator<` (strict weak ordering).

std::set — sorted unique elements

```
#include <set>

std::set<int> s = {5, 2, 3, 2, 7}; // {2, 3, 5, 7} (dup removed)

s.insert(4); // {2, 3, 4, 5, 7}
s.erase(3); // {2, 4, 5, 7}
s.count(5); // 1 (present)
s.count(99); // 0 (absent)

auto it = s.find(4); // iterator to element 4
if (it != s.end())
    std::cout << *it; // prints 4

// iteration visits in sorted order
for (const auto& x : s)
    std::cout << x << " "; // 2 4 5 7
```

std::map — sorted key-value pairs

```
#include <map>

std::map<std::string, int> ages;
ages["Alice"] = 20;
ages["Bob"] = 22;
ages["Carol"] = 21;

ages["Alice"]; // 20
ages.at("Bob"); // 22 (throws if key missing)

ages["Dave"] = 19; // insert new pair
ages["Alice"] = 21; // update existing value

ages.count("Eve"); // 0

// Danger: operator[] inserts a default if key is absent!
std::cout << ages["Eve"]; // prints 0, "Eve" now in the map!
ages.size(); // 5 (not 4!)
```

Safe lookup: use `find()` or `count()` before `operator[]` to avoid accidental insertion.

std::map — searching and iterating

```
std::map<std::string, int> ages = {
    {"Alice", 20}, {"Bob", 22}, {"Carol", 21}
};

// find returns iterator to std::pair<const Key, Value>
auto it = ages.find("Bob");
if (it != ages.end()) {
    std::cout << it->first; // "Bob"
    std::cout << it->second; // 22
}

// iteration visits keys in sorted order
for (auto it = ages.begin(); it != ages.end(); ++it) {
    std::cout << it->first << ": " << it->second << "\n";
}
// Output: Alice: 20 Bob: 22 Carol: 21
```

C++17 structured bindings make this cleaner:

```
for (const auto& [name, age] : ages)
    std::cout << name << ": " << age << "\n";
```

std::multiset — duplicates allowed

```
#include <set> // multiset lives here too

std::multiset<int> ms = {1, 2, 5, 2, 2}; // {1, 2, 2, 2, 5}

ms.insert(5); // {1, 2, 2, 2, 5, 5}

ms.count(2); // 3 (not just 0 or 1!)

auto it = ms.find(5); // iterator to *one* of the 5s
auto [lo, hi] = ms.equal_range(2);
// [lo, hi) spans all elements equal to 2
for (auto i = lo; i != hi; ++i)
    std::cout << *i << " "; // 2 2 2
```

Same interface as `set` except: `insert` always succeeds (duplicates OK), and `count` can return values > 1.

Associative container complexity

	`insert`	`erase`	`find`	`count`	`lower/upper`
Ordered	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
Unordered	$O(1)^*$	$O(1)^*$	$O(1)^*$	$O(1)^*$	—

* Average case. Worst case is $O(n)$ due to hash collisions.

Ordered containers also provide `lower_bound` and `upper_bound`; unordered containers do not (elements are not sorted).

Unordered associative containers (C++11)

Hash-table-based containers with $O(1)$ average lookup:

- `std::unordered_set<K>` — unique keys, hashed
- `std::unordered_map<K,V>` — unique key–value pairs, hashed
- `std::unordered_multiset<K>` — duplicate keys, hashed
- `std::unordered_multimap<K,V>` — duplicate key–value pairs, hashed

Keys must support `std::hash<K>` and `operator==`.
The standard provides hash for built-in types and `std::string`.

When to prefer over ordered:

- You don't need sorted iteration.
- You want faster average lookup ($O(1)$ vs. $O(\log n)$).

When to prefer ordered:

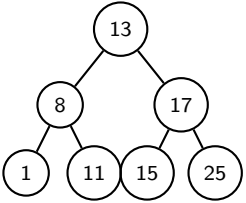
- You need sorted iteration or `lower_bound`/`upper_bound`.
- Worst-case $O(\log n)$ is acceptable; $O(n)$ worst case is not.

std::unordered_map — example

```
#include <unordered_map>
#include <string>

std::unordered_map<std::string, int> freq;
std::string word;
while (std::cin >> word)
    freq[word]++; // default-inserts 0, then increments

// iterate (order is NOT sorted!)
for (const auto& [w, count] : freq)
    std::cout << w << ": " << count << "\n";

// lookup
auto it = freq.find("hello");
if (it != freq.end())
    std::cout << it->second;
```

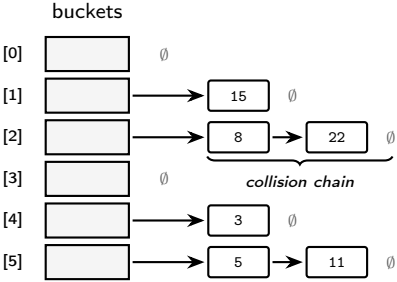
Internal representation

Ordered (red-black tree):



Elements always in sorted order.
All ops $O(\log n)$ guaranteed.

Unordered (hash table with chaining):



No sorted order. `hash(key) % bucket_count`
$O(1)$ average, $O(n)$ worst case (long chain).

Container adaptors: `stack`, `queue`, `priority_queue`

Adaptors wrap an existing container and **restrict** its interface:

```
#include <stack>
#include <queue>

std::stack<int> stk; // default: wraps std::deque
stk.push(1); stk.push(4); // [1, 4]
stk.top(); // 4 (LIFO)
stk.pop(); // [1]

std::queue<int> q; // default: wraps std::deque
q.push(1); q.push(4); // [1, 4]
q.front(); // 1 (FIFO)
q.pop(); // [4]

std::priority_queue<int> pq; // default: wraps std::vector
pq.push(1); pq.push(5); pq.push(3);
pq.top(); // 5 (max element)
pq.pop(); // removes 5, top is now 3
```

No iterators, no `operator[]`, no `begin()`/`end()`. Access *only* through the restricted interface.

Container adaptor summary

Adaptor	Policy	Key methods	Default underlying
`stack`	LIFO	`push`, `pop`, `top`	`deque`
`queue`	FIFO	`push`, `pop`, `front`	`deque`
`priority_queue`	Max-heap	`push`, `pop`, `top`	`vector`

You can specify a different underlying container:

```
std::stack<int, std::vector<int>> stk;
```

Key idea: restrict operations to enforce a specific access pattern. This is *encapsulation at the data-structure level*.

What is an iterator?

An **iterator** is a generalised pointer that lets you traverse a container without knowing its internal structure.

```
std::vector<int> v = {10, 20, 30};

std::vector<int>::iterator it = v.begin();

*it; // 10 (dereference --- read the element)
++it; // advance to next element
*it; // 20
++it;
*it; // 30
++it;
// it == v.end() --- one past the last element (sentinel)
```

Every STL container provides `begin()` (first element) and `end()` (one-past-the-last sentinel).

A **pointer** is the simplest iterator — it satisfies all iterator requirements.

Iterating with a `for` loop

The canonical iterator loop:

```
std::vector<int> v = {10, 20, 30, 40};

for (std::vector<int>::iterator it = v.begin();
     it != v.end(); ++it)
{
    std::cout << *it << " "; // 10 20 30 40
}
```

With `auto` (C++11), this becomes much cleaner:

```
for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << " ";
```

Key pattern: `begin()` → first element; `end()` → one-past-last; `++it` → advance; `*it` → access.

Range-based `for` — syntactic sugar

C++11 provides a shorthand that the compiler rewrites into an iterator loop:

```
std::vector<int> v = {10, 20, 30, 40};

for (const auto& x : v)
    std::cout << x << " "; // 10 20 30 40
```

The compiler translates this into roughly:

```
{
    auto __begin = v.begin();
    auto __end = v.end();
    for ( ; __begin != __end; ++__begin) {
        const auto& x = *__begin;
        std::cout << x << " ";
    }
}
```

Requirement: the container must provide `begin()` and `end()`.

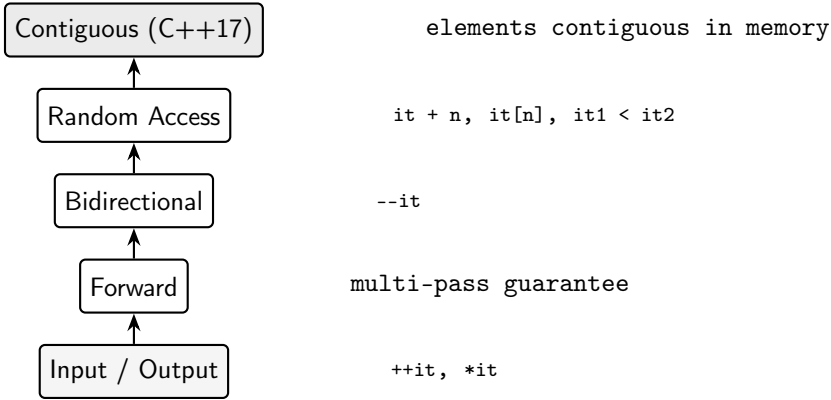
Iterators work uniformly across containers

The *same* loop pattern works for every STL container:

```
// vector
std::vector<int> v = {3, 1, 4};
for (auto it = v.begin(); it != v.end(); ++it)
    std::cout << *it << " "; // 3 1 4

// set (sorted order)
std::set<int> s = {3, 1, 4, 1};
for (auto it = s.begin(); it != s.end(); ++it)
    std::cout << *it << " "; // 1 3 4

// map (sorted by key)
std::map<std::string, int> m = {{"b",2}, {"a",1}};
for (auto it = m.begin(); it != m.end(); ++it)
    std::cout << it->first << ":" << it->second << " ";
                                    // a:1 b:2

// list
std::list<int> lst = {9, 7, 5};
for (auto it = lst.begin(); it != lst.end(); ++it)
    std::cout << *it << " "; // 9 7 5
```

Iterator categories

Not all iterators support the same operations. C++ defines a hierarchy:



Each category *adds* operations to the one below it.

Iterator semantics — what each category requires

Category	Required operations
Input / Output	`*it`, `++it`, `it++`, `==`, `!=`
Forward	+ default constructible, multi-pass
Bidirectional	+ `--it`, `it--`
Random Access	+ `it + n`, `it - n`, `it[n]`, `<`, `>`, `<=`, `>=`
Contiguous	+ elements adjacent in memory

Why it matters: algorithms declare their *minimum* iterator requirement. If your container's iterators don't meet it, the code won't compile.

Which containers give which iterators?

Container	Iterator category
`std::array`	Contiguous / Random Access
`std::vector`	Contiguous / Random Access
`std::deque`	Random Access
`std::list`	Bidirectional
`std::forward_list`	Forward
`std::set / map`	Bidirectional
`std::unordered_*`	Forward
raw array / pointer	Contiguous / Random Access

Example consequence: `std::sort` requires *random access* iterators. You can sort a `vector` but **not** a `list` with `std::sort`. (A `list` provides its own `.sort()` member.)

Random access vs. bidirectional — a concrete example

```
#include <vector>
#include <list>

std::vector<int> v = {10, 20, 30, 40, 50};
auto vit = v.begin();
vit += 3; // OK: random access
std::cout << *vit; // 40

std::list<int> lst = {10, 20, 30, 40, 50};
auto lit = lst.begin();
// lit += 3; // COMPILER ERROR -- list is bidirectional only
++lit; ++lit; ++lit; // OK: advance one step at a time
std::cout << *lit; // 40
```

For bidirectional/forward iterators, use `std::advance`:

```
#include <iterator>
auto lit2 = lst.begin();
std::advance(lit2, 3); // moves lit2 forward by 3 steps
std::cout << *lit2; // 40
```

Iterator flavours: const, reverse

Every container provides four iterator types:

```
std::vector<int> v = {10, 20, 30};

// regular iterator (read + write)
for (auto it = v.begin(); it != v.end(); ++it)
    *it *= 2; // {20, 40, 60}

// const iterator (read only)
for (auto it = v.cbegin(); it != v.cend(); ++it)
    std::cout << *it << " "; // 20 40 60
    // *it = 0; // COMPILER ERROR

// reverse iterator
for (auto it = v.rbegin(); it != v.rend(); ++it)
    std::cout << *it << " "; // 60 40 20

// const reverse iterator
for (auto it = v.crbegin(); it != v.crend(); ++it)
    std::cout << *it << " "; // 60 40 20
```

Best practice: prefer `cbegin()`/`cend()` when you don't need to modify elements.

Free-standing iterator access (C++11/14)

`<iterator>` provides free functions that work on containers, plain arrays, and initialiser lists:

```
#include <iterator>

int arr[] = {10, 20, 30};

// works on plain arrays!
auto it = std::begin(arr); // pointer to arr[0]
auto e = std::end(arr); // pointer past arr[2]

// C++14: const versions
auto cit = std::cbegin(arr); // const pointer
auto cre = std::crend(arr); // const reverse

// works on any container too
std::vector<int> v = {1, 2, 3};
auto vb = std::begin(v); // same as v.begin()
```

These enable writing generic code that works on *both* containers and raw arrays.

Iterator utility functions (1/2)

`<iterator>` provides helpers that adapt to the iterator category automatically:

```
#include <iterator>
#include <vector>
#include <list>

std::vector<int> v = {10, 20, 30, 40};

auto it1 = std::next(v.begin(), 2); // points to 30
auto it2 = std::prev(v.end(), 1); // points to 40
int d = std::distance(it1, it2); // 1

std::list<int> lst = {10, 20, 30, 40};
auto lit = lst.begin();
std::advance(lit, 2); // lit now points to 30
int d2 = std::distance(lst.begin(), lit); // 2
```

Iterator utility functions (2/2)

Function	Effect
`std::next(it, n)`	Returns iterator `n` steps forward
`std::prev(it, n)`	Returns iterator `n` steps backward
`std::advance(it, n)`	Modifies `it` in place
`std::distance(a, b)`	Number of increments from `a` to `b`

Iterator invalidation — a common trap

Modifying a container while iterating can invalidate iterators:

```
std::vector<int> v = {1, 2, 3, 4, 5};

for (auto it = v.begin(); it != v.end(); ++it) {
    v.push_back(*it); // UNDEFINED BEHAVIOUR!
}
```

`push_back` may reallocate the vector's storage, making `it` point to freed memory.

Rule: Never modify a container's *size* while holding active iterators to it.

Read-only operations (e.g., `find`, `at`) never invalidate iterators.

Iterator invalidation rules by container

Container	Insertion invalidates...	Erasure invalidates...
`vector`	All iterators if reallocation; otherwise only those at/after insertion point	Iterators at/after erased element
`deque`	All iterators (insertion at front or back may not invalidate, but references may)	All iterators (erase at ends: only the erased element's iterator)
`list`	None	Only the erased element's iterator
`set/map`	None	Only the erased element's iterator
`unordered_*`	All if rehash; otherwise none	Only the erased element's iterator

`std::list` and `std::set/map` have the *strongest* iterator stability guarantees.

The algorithms library

`#include <algorithm>` provides 100+ functions that operate on iterator ranges `[begin, end)`:

```
#include <algorithm>
#include <vector>

std::vector<int> v = {7, 2, 9, 4, 1};

std::sort(v.begin(), v.end()); // {1, 2, 4, 7, 9}

auto it = std::find(v.begin(), v.end(), 4);
if (it != v.end())
    std::cout << "Found at position "
              << (it - v.begin()); // "Found at position 2"

int cnt = std::count(v.begin(), v.end(), 7); // 1

bool has3 = std::binary_search(
    v.begin(), v.end(), 3); // false (v is sorted)
```

How `std::find` works internally

The implementation reveals the iterator pattern:

```
template<class InputIt, class T>
InputIt find(InputIt first, InputIt last, const T& value) {
    for ( ; first != last; ++first) {
        if (*first == value)
            return first;
    }
    return last; // not found -> return end
}
```

Notice: no mention of `vector`, `list`, or `set`. The algorithm only uses:

`++first` (advance), `first != last` (compare), `*first` (dereference).

Any container whose iterators support these three operations can use `std::find`.

Non-modifying algorithms

```
#include <algorithm>
#include <numeric>
#include <vector>

std::vector<int> v = {7, 2, 9, 4, 1};

// search
auto it = std::find(v.begin(), v.end(), 9); // iterator to 9
int cnt = std::count(v.begin(), v.end(), 2); // 1

// min / max
auto minIt = std::min_element(v.begin(), v.end()); // -> 1
auto maxIt = std::max_element(v.begin(), v.end()); // -> 9
int minPos = minIt - v.begin(); // 4

// equality
std::vector<int> w = {7, 2, 9, 4, 1};
bool same = std::equal(v.begin(), v.end(), w.begin()); // true

// accumulate (in <numeric>)
int sum = std::accumulate(v.begin(), v.end(), 0); // 23
```

`min_element` / `max_element` return *iterators*, not values.

Modifying algorithms

```
#include <algorithm>
#include <numeric>
#include <vector>

std::vector<int> v = {5, 3, 8, 1, 4};

std::sort(v.begin(), v.end()); // {1, 3, 4, 5, 8}
std::reverse(v.begin(), v.end()); // {8, 5, 4, 3, 1}
std::fill(v.begin(), v.end(), 0); // {0, 0, 0, 0, 0}
std::iota(v.begin(), v.end(), 10); // {10, 11, 12, 13, 14}
```

Predict: What does this produce?

```
std::vector<int> w = {3, 1, 4, 1, 5};
std::sort(w.begin(), w.end());
auto last = std::unique(w.begin(), w.end());
// What are the elements in [w.begin(), last)? ???
```

Answer: `{1, 3, 4, 5}` — the sorted unique values.

`std::unique` only removes *consecutive* duplicates, so sort first!

Function objects (functors) as predicates

Many algorithms accept a callable to customise their behaviour:

```
#include <algorithm>
#include <vector>

struct Descending {
    bool operator()(int a, int b) const {
        return a > b;
    }
};

std::vector<int> v = {7, 2, 9, 4};

// default sort: ascending
std::sort(v.begin(), v.end()); // {2, 4, 7, 9}

// custom sort: descending (pass a functor instance)
std::sort(v.begin(), v.end(), Descending()); // {9, 7, 4, 2}
```

A **functor** is any class that overloads `operator()`. The algorithm calls `cmp(a, b)` just as if it were a plain function.

find_if with a functor

`std::find_if` returns an iterator to the *first* element satisfying a predicate:

```
#include <algorithm>
#include <vector>

struct InRange {
    bool operator()(int value) const {
        return value >= 3 && value <= 6;
    }
};

std::vector<int> v = {7, 2, 9, 4, 1};

auto it = std::find_if(v.begin(), v.end(), InRange());

if (it != v.end())
    std::cout << *it; // 4 (first element in [3,6])
```

Predict: If we change `v` to `{7, 8, 9}`, what does `find_if` return? → `v.end()` (not found)

The erase-remove idiom

`std::remove` does not actually shrink the container — it moves unwanted elements to the end and returns a new logical end:

```
#include <algorithm>
#include <vector>

std::vector<int> v = {1, 2, 3, 2, 4, 2, 5};

// step 1: "remove" all 2s (logical removal only)
auto newEnd = std::remove(v.begin(), v.end(), 2);
// v is now: {1, 3, 4, 5, ?, ?, ?} size still 7!

// step 2: actually erase the tail
v.erase(newEnd, v.end());
// v is now: {1, 3, 4, 5} size is 4
```

This two-step pattern is called the **erase-remove idiom**. It exists because algorithms operate on iterator ranges and have *no knowledge of the container itself*.

Algorithms on plain arrays

STL algorithms work on raw arrays too — a pointer is an iterator:

```
#include <algorithm>
#include <numeric>

int arr[] = {7, 6, -1, 6, 3};

std::sort(arr, arr + 5); // {-1, 3, 6, 6, 7}

auto last = std::unique(arr, arr + 5);
// arr: {-1, 3, 6, 7, ...} (4 unique elements)
int n = last - arr; // 4

int sum = std::accumulate(arr, arr + n, 0); // 15

std::iota(arr, arr + 5, 2); // {2, 3, 4, 5, 6}
```

Pass `arr` as begin and `arr + N` as end. A pointer satisfies the contiguous/random-access iterator requirements.

Algorithm Ref sheet

Category	Function	What it does
Search	`find`, `find_if`	First match / predicate match
	`count`, `count_if`	Count matches
	`binary_search`	Boolean: is value in sorted range?
	`lower_bound`, `upper_bound`	Iterator to insertion point
Sort	`sort`, `stable_sort`	Sort range
	`partial_sort`	Sort first k elements
	`nth_element`	Partition around n-th element
Modify	`reverse`, `fill`, `iota`	Reorder / overwrite
	`copy`, `swap_ranges`	Copy / swap
	`remove`, `remove_if`	Logical removal (use erase-remove)
	`unique`	Remove consecutive duplicates
Numeric	`accumulate`	Sum / fold
	`partial_sum`	Prefix sums
Min/Max	`min_element`, `max_element`	Iterator to min / max

All in `<algorithm>` except `accumulate`, `iota`, `partial_sum` which are in `<numeric>`.

Structured bindings (C++17)

Decompose a `pair`, `tuple`, or struct into named variables:

```
// Before C++17
auto it = myMap.find("key");
if (it != myMap.end())
    std::cout << it->first << ": " << it->second;

// C++17: structured bindings
std::map<std::string, int> ages = {{"Alice",20}, {"Bob",22}};

for (const auto& [name, age] : ages)
    std::cout << name << ": " << age << "\n";
```

Also works with `insert` results:

```
auto [iter, success] = ages.insert({"Carol", 21});
if (success)
    std::cout << "Inserted " << iter->first;
```

Decision guide: which container?

If you need...	**Use**
Fast random access + dynamic size	`vector`
Fast insert/remove at both ends	`deque`
Fast insert/remove anywhere (with iterator)	`list`
Fixed-size array with value semantics	`array`
Sorted unique keys	`set`
Sorted key-value lookup	`map`
Fast average-case lookup (no ordering)	`unordered_map` / `unordered_set`
LIFO access pattern	`stack`
FIFO access pattern	`queue`
Always access the largest/smallest	`priority_queue`

When in doubt: start with `std::vector`. Its cache-friendly memory layout often outperforms theoretically better containers.

References

- **Federico Busato**, *Modern C++ Programming*, Ch. 20: Containers, Iterators, and Algorithms (2026)
- **Bjarne Stroustrup**, *The C++ Programming Language*, 4th ed., Ch. 31–33 (STL Containers, Algorithms)
- **Nicolai Josuttis**, *The C++ Standard Library*, 2nd ed. (2012)
- **cppreference.com** — Containers library, Algorithms library, Iterator library
- **Arthur O'Dwyer**, *Mastering the C++17 STL* (2017)